

摘 要

本作品完成了一个无源交流电流表及无线读表器，该表由变压器，电流互感器，电磁耦合供电模块，TI MSPM0G507 单片机四部分构成。

关键词：无线读表器；电流互感器；

1 程序设计

1.1 软件总体结构

软件以测量周期为单位循环执行，流程见图 1。每周期依次完成探测帧采集与量程判定、测量帧采集、有效值计算、标定换算、显示与无线发送。采样由定时器触发 ADC、DMA 成帧，采样间隔由时钟决定。主要参数见表 1。

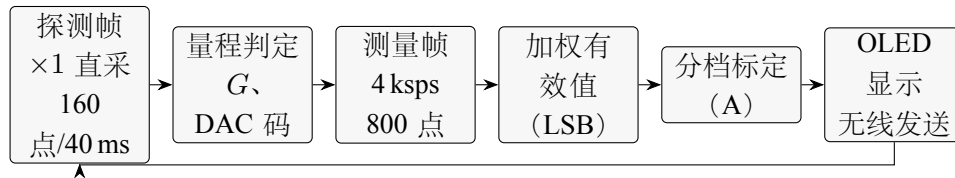


图 1 测量周期流程

表 1 采样与测量参数

参数	取值
采样频率 f_s	4 kHz
帧长 N	800 点 (200 ms, 10 个工频周期)
探测帧长	160 点 (40 ms)
ADC	12 位, 参考片内 2.5 V 基准, 满量程 0–4095
DAC	12 位, 与 ADC 共用同一基准
增益档位	×1、×2、×4、×8、×16、×32
窗函数	周期型 Hann

1.2 交流电流有效值的数字测量

交流电流有效值定义为 $I = \sqrt{\frac{1}{T} \int_0^T i^2(t) dt}$ 。对等间隔采样序列 $x[n]$ ，取整数枢轴 p 、窗函数 $w[n] = \frac{1}{2}(1 - \cos \frac{2\pi n}{N})$ ，加权直流与加权有效值为

$$\delta = \frac{\sum_n w[n] (x[n] - p)}{\sum_n w[n]}, \quad X_{\text{rms}} = \sqrt{\frac{\sum_n w[n] (x[n] - p - \delta)^2}{\sum_n w[n]}}. \quad (1)$$

帧长取工频周期的整数倍， $N = f_s M / f = 800$ ($M = 10$, $f = 50$ Hz)。误差按下列各项估算，汇总于表 2。

非同步采样误差 工频与采样时钟均存在偏差，实际帧长不严格为整周期，平方和中残留二倍频分量，其幅度等于窗序列频响在 $2f$ 处的取值。本设计 $2f$ 对应第 20 号频点：矩形窗该处旁瓣约 -35 dB，按 49.66 Hz 计对应有效值误差 0.33 %；Hann 窗低于 -95 dB，该项可忽略。 h 次谐波对应第 $20h$ 号频点，抑制更深。因此本设计不需锁频与过零检测。

量化误差 量化噪声有效值 $q/\sqrt{12} = 0.29 \text{ LSB}$ 。0.1 A 在 $\times 32$ 档的读数约 233 LSB，量化项为 0.12 %；不加放大时约 7 LSB，量化项达 4 %。

噪声偏置 噪声与信号正交叠加，读数为 $\sqrt{X^2 + \sigma^2}$ ，相对偏差 $\sigma^2/2X^2$ ，为单边误差，多帧平均不能消除。该项由前端纹波决定，需实测 σ 后确定。

表 2 有效值测量误差预算 (0.1 A, $\times 32$ 档, 指标 0.5 %)

误差项	量值	来源
非同步采样 (Hann 窗)	$< 0.01 \%$	分析值, 矩形窗为 0.33 %
量化	0.12 %	0.29 LSB / 233 LSB
单帧噪声底	0.043 %	分析值, 加窗前 0.035 %
噪声偏置 $\sigma^2/2X^2$	$\sigma = 1 \text{ LSB}$ 时 0.06 %; $\sigma = 3 \text{ LSB}$ 时 0.53 %	待实测 σ
标定表插值残差	待实测	由标定点间距决定

1.3 自动量程与偏置

0.1–2 A 量程比为 20:1，指标 0.5 % 在下端为 0.5 mA，故采用逐周期自动量程。片内运放配置为非反相可编程增益放大器，电阻梯底部接片内 12 位 DAC，输出电压、偏置码与选档条件为

$$V_{\text{out}} = G V_{\text{in}} + (1 - G) V_{\text{dac}}, \quad D = \frac{Gm - C}{G - 1}, \quad \frac{Gp}{2} \leq 0.65 \min(d, 4095 - d), \quad (2)$$

式中 m 、 p 为探测帧测得的输入直流与峰峰值， $C = 2047$ 为窗口中心， d 为该档输出直流，取满足条件的最大档。DAC 使放大以输入直流为支点；直流灵敏度 $dV_{\text{out}}/dV_{\text{in,dc}} = G$ ， $\times 32$ 档上 50 mV 失配即使输出偏离中心 1.6 V。 $\times 1$ 档由 ADC 直接读取输入，用于输入直流过低、各 PGA 档均无法容纳的情形。换档设 25 %/75 % 回差。

1.4 标定

标定表为“加权有效值 (LSB) $\rightarrow$ 电流 (A)”的分段线性表，每档一张，读数按 $I = y_0 + (y_1 - y_0)(X - x_0)/(x_1 - x_0)$ 插值，表外按端段外推。分档建表的原因是片内电阻梯各档实际比值对标称 2/4/8/16/32 的偏离与 0.5 % 指标同量级，且互感器在低磁通下比差随电流变化，单一比例系数无法覆盖。标定数据由台架比对获得，填表方法见测试章节。

1.5 通信协议

无线链路为透传串口，不提供分包与长度字段，故采用面向行的 ASCII 协议：以换行为自同步定界符，接收端在任意位置接入或丢失字节后都能在下一个换行重新对齐；

不合格式的行整行丢弃。链路速率 115200 bps，一次读数的两行帧约 110 字节，线上时间不足 10 ms。

题目 2(2) 规定两种读取模式下均不得操作电流表，故读数由读表器请求产生，电流表不主动上报测量值。帧格式见表 3。

表 3 帧格式

方向	帧	说明
读表器 → 电流表	MEAS	广播，任一在线电流表应答
读表器 → 电流表	MEAS, ADDR= <i>n</i>	仅编码开关为 <i>n</i> 的电流表应答
电流表 → 读表器	METER_TEST, ADDR= <i>n</i> , CURRENT_MA= <i>m</i> , STATUS= <i>s</i>	读数帧。 <i>s</i> ∈ {NORMAL, LOW, HIGH}，按 0.2 A/2 A 判定
电流表 → 读表器	METER_CAL, ADDR= <i>n</i> , RMS= <i>x</i> , GAIN= <i>g</i> , FLAG= <i>f</i> , ...	同次读数的原始加权有效值、档位与质量标志，供标定与诊断
电流表 → 读表器	IMHERE, ADDR= <i>n</i>	保活心跳，空闲时每 2 s 一次，不含读数

命令带地址而非仅用广播，是因为一台电流表在不同时刻由编码开关赋予不同地址，仅凭连接无法确定对端身份；带地址后请求与应答一一对应，不会把某地址的应答记到另一地址名下。

心跳不参与测量，不唤醒任何模拟电路，其作用有二：编码开关改变时旧地址的心跳停止、新地址出现，读表器据此更新在场地址；负载断开后电流表随之失电，心跳消失，即为题目 2(3) 的离线判据，较等待一次请求超时更快。

读数不可信时（基准未就绪、输入越出转换窗口、采集帧未完成）电流表只发 METER_CAL 并在 FLAG 中给出原因，不发读数帧，避免把不可信的数值以正常状态送出。

1.6 读表器软件

读表器周期扫描（8 s 扫描、12 s 间歇）并对搜索到的电流表建立连接，连接建立后不再持续收数：读数由请求产生，链路空闲是常态，故链路健康仅由连接状态判定，不以数据超时判定。

基本模式为一对一手动读取，操作读表器选定地址后发出 MEAS, ADDR=*n*，收到读数帧即显示并记录。自动模式一键启动，每 2 分钟对当前在场（心跳可闻）的地址依次发出请求；同一连接上同时只允许一个未完成请求，故轮次内串行执行。

在场与离线由心跳判定：收到该地址任一帧即刷新其在场时刻，连续 7 s（三个心跳周期）无帧则判为离线并给出指示。离线只作指示，不写入记录——记录用于保存读数，而缺席不是读数。请求超时同样不写入记录，只提示本次未读到，且不据此判定离线：正在心跳的电流表只是本次未及应答，并非不在。

报警按读数判定，低于 0.2 A 为低限、高于 2 A 为超限，每一次超限读数均给出声音与振动提示，不以状态跳变为条件——读数稀疏，同一状态的相邻两次读数是两个独立事件。

记录存入手机数据库，含读取时间、电流表地址、电流值与状态，掉电不丢失，容量 1000 条，并按地址绘制电流趋势曲线。

A 附录 B 有效值计算源码

一帧 800 点的 Hann 加权真有效值。均值与平方和用同一个窗加权，并分两趟：合成一趟要用 $\overline{wx^2} - \bar{x}^2$ ，是两个 4.2×10^6 量级的数相减去取 10^2 量级的结果，单精度浮点的尾数不够这次抵消。

Listing 1 Hann 加权真有效值

```
1 // Pass 1: Hann-weighted DC, as an offset from the integer pivot.
2 let (mut sum_w, mut sum_wd) = (0.0f32, 0.0f32);
3 for (&x, &w) in buf.iter().zip(HANN.iter()) {
4     sum_w += w;
5     sum_wd += w * ((x & 0xFFFF) as i32 - pivot) as f32;
6 }
7 let offset = sum_wd / sum_w;
8
9 // Pass 2: weighted mean square of the residual.
10 let mut sum_wd2 = 0.0f32;
11 for (&x, &w) in buf.iter().zip(HANN.iter()) {
12     let d = ((x & 0xFFFF) as i32 - pivot) as f32 - offset;
13     sum_wd2 += w * d * d;
14 }
15 sqrtf(sum_wd2 / sum_w) // rms, in ADC LSB
```